

Roteiro de narração para os vídeos gravados

Projeto Vigilância EPI | protótipo e estrutura técnica

Como usar

- Leia como se estivesse explicando para a banca, não como se estivesse lendo um texto decorado.
- No editor de vídeo, deixe o áudio original bem baixo ou mudo e grave sua voz por cima acompanhando os tempos.
- Se a cena passar antes de você terminar um bloco, corte a última frase do bloco e siga para o próximo.
- Se possível, corte os segundos iniciais e finais em que aparece apenas o OBS. Se não cortar, use as falas de abertura e fechamento como estão.
- No primeiro vídeo, fale como demonstração do fluxo do protótipo. Não force a ideia de detecção ao vivo se a tela estiver apenas mostrando streams aguardando imagem.
- No segundo vídeo, não tente explicar todos os arquivos. O objetivo é mostrar organização técnica: frontend, backend, dados, modelos, scripts e deploy.

Vídeo 1 - Demonstração do protótipo

Arquivo: C:\Users\andre\Videos\OBS Studio\2026-06-08 00-02-39.mp4

Duração aproximada: 8min31s.

Ritmo: fala calma, com pequenas pausas nas trocas de tela.

00:00 a 00:20 - Abertura

Na tela: Aparece o OBS e o início da gravação.

Neste vídeo eu vou demonstrar o protótipo do Vigilância EPI, que é o sistema que eu desenvolvi para apoiar a fiscalização do uso de equipamentos de proteção. A ideia aqui é mostrar o fluxo funcionando: abrir o sistema, configurar as partes principais e explicar como ele foi pensado para uso em um ambiente real.

00:20 a 00:55 - Preparação do ambiente

Na tela: Você alterna entre janelas e prepara a execução do projeto.

No começo eu deixo o ambiente pronto para a demonstração. O projeto tem uma parte web, que é a interface usada pelo usuário, e uma parte em Python, que fica responsável pela API, pelos dados e pelo processamento das imagens.

Esse início é mais técnico, mas ele é importante porque mostra que o protótipo não é só uma tela desenhada. Ele está rodando com backend, frontend, arquivos locais e serviços separados.

00:55 a 01:35 - Inicialização do backend

Na tela: Terminal iniciando o sistema e carregando configurações.

Aqui eu inicio o backend. É essa parte que recebe as requisições da interface, controla os usuários, acessa os dados e prepara a comunicação com os modelos de visão computacional.

Durante essa etapa aparecem os logs de execução. Eles servem para acompanhar se o servidor subiu corretamente, se as rotas foram carregadas e se o sistema está pronto para receber acesso pelo navegador.

01:35 a 02:20 - Servidor pronto

Na tela: Terminal com banner, mensagens e links de acesso.

Com o servidor rodando, o protótipo já pode ser acessado pelo navegador. Em uma versão final, essa parte ficaria escondida do usuário comum, mas para a apresentação eu mostro porque ela comprova que existe uma aplicação real executando por trás da interface.

Também é nessa camada que entram recursos como múltiplos streams, modelos personalizados, controle de usuários e integração com um hub de backends.

02:20 a 03:00 - Login

Na tela: Tela de entrada do Vigilância EPI.

Agora eu entro na interface do Vigilância EPI. O sistema usa login para separar os dados de cada usuário. Isso é importante porque uma empresa não deve misturar câmeras, funcionários, modelos e cadastros com os dados de outra conta.

Nesta tela o usuário pode entrar com e-mail e senha ou ir para o cadastro, caso ainda não tenha uma conta.

03:00 a 03:35 - Cadastro de usuário

Na tela: Tela de cadastro com campos de nome, e-mail e senha.

Aqui eu mostro o cadastro. Para o protótipo, o objetivo é permitir que cada usuário tenha seu próprio ambiente de testes. Depois de criar a conta, o sistema passa a organizar os dados em uma área separada.

Essa separação foi pensada porque o projeto lida com informações sensíveis, como imagens, funcionários e configurações de monitoramento.

03:35 a 04:15 - Painel principal

Na tela: Interface principal com abas de câmeras, EPIs, funcionários, dispositivos, modelos e configuração.

Depois do login, aparece o painel principal. No topo ficam as principais áreas do sistema: câmeras, EPIs, funcionários, dispositivos, modelos e configurações.

A aba de câmeras é o ponto central da demonstração, porque é nela que o usuário conecta uma fonte de vídeo, escolhe um modelo e acompanha o estado do monitoramento.

04:15 a 05:15 - Cadastro e visualização de streams

Na tela: Tela de câmeras, botão de adicionar câmera de rede e cards de streams.

Nessa parte eu mostro como o sistema trabalha com streams. O usuário pode adicionar uma câmera de rede ou uma câmera manual, e cada fonte aparece como um card separado.

A proposta é permitir mais de uma câmera no mesmo painel. Assim, em vez de depender de uma única webcam, o protótipo consegue representar melhor um cenário de empresa, onde poderiam existir várias áreas sendo acompanhadas ao mesmo tempo.

Mesmo quando algum stream ainda está aguardando imagem, o painel já mostra a estrutura do monitoramento, com controles e espaço para o processamento de cada câmera.

05:15 a 05:55 - Cadastro de EPIs

Na tela: Aba de EPIs com cadastro, fotos ou opções relacionadas ao treinamento.

Na aba de EPIs ficam os equipamentos que a empresa quer acompanhar. Pode ser capacete, colete, óculos, uniforme ou outro item usado no ambiente de trabalho.

Além do cadastro do nome do EPI, o sistema permite associar imagens. Essas imagens podem ser usadas depois para melhorar ou treinar um modelo específico para a realidade da empresa.

05:55 a 06:40 - Funcionários

Na tela: Aba de funcionários e modal de novo funcionário.

Aqui eu mostro a parte de funcionários. O cadastro guarda informações como nome, cargo e matrícula, e também pode receber fotos de rosto.

No protótipo, essa parte funciona como uma prova de conceito para reconhecimento de funcionários. Em uma aplicação real, esse módulo precisaria seguir regras bem claras de autorização, privacidade e proteção dos dados.

Mesmo assim, ele mostra que o sistema pode relacionar a detecção de EPI com informações de quem está no ambiente monitorado.

06:40 a 07:30 - Modelos de detecção

Na tela: Aba de modelos com pesos YOLO, modelos treinados e classes detectadas.

Na aba de modelos ficam os modelos de detecção disponíveis. O sistema pode usar modelos base, modelos enviados pelo usuário ou modelos treinados a partir dos dados cadastrados.

Cada modelo tem suas classes detectadas. A partir disso, o usuário define quais itens são obrigatórios em uma leitura, por exemplo capacete, colete ou outro EPI.

Essa parte é importante porque o sistema não fica preso a uma regra única. Ele pode ser ajustado conforme o ambiente de trabalho e conforme os equipamentos que realmente precisam ser fiscalizados.

07:30 a 08:05 - Modelo personalizado e desempenho

Na tela: Cards de modelos, status e opção de uso de modelo.

Também aparecem modelos personalizados e informações de status. Em máquinas com placa NVIDIA, a arquitetura prevê o uso de TensorRT para melhorar desempenho.

Isso foi pensado para quando o sistema precisar processar mais câmeras ou trabalhar com modelos maiores. No protótipo, eu deixo essa parte visível para mostrar que existe uma preocupação com escalabilidade, e não apenas com a tela do usuário.

08:05 a 08:30 - Configurações

Na tela: Aba de configuração com backend, modo de seleção e dados da conta.

Por fim, eu mostro a tela de configuração. Nela é possível ajustar o backend, atualizar a comunicação com o hub e escolher o modo de seleção do servidor.

Essa separação permite que o frontend rode em um lugar e o processamento pesado rode em outro, inclusive em uma máquina com GPU. Na prática, isso deixa o projeto mais flexível para crescer.

08:30 a 08:31 - Fechamento do trecho

Na tela: Volta para o OBS no final da gravação.

Com isso, o primeiro vídeo mostra o fluxo principal do protótipo: iniciar o backend, acessar a interface, cadastrar dados, preparar câmeras, organizar modelos e configurar a comunicação do sistema.

Vídeo 2 - Arquivos do sistema e parte técnica

Arquivo: C:\Users\andre\Videos\OBS Studio\2026-06-08 00-12-19.mp4

Duração aproximada: 1min41s.

Ritmo: mais direto, como explicação técnica complementar.

00:00 a 00:08 - Introdução do segundo vídeo

Na tela: Aparece o OBS no início.

Neste segundo vídeo eu mostro rapidamente a parte técnica do projeto, principalmente os arquivos e a estrutura usada para o sistema funcionar.

00:08 a 00:20 - Interface web

Na tela: Tela de login do sistema no navegador.

Eu começo pela interface web para situar o que foi mostrado no vídeo anterior. Essa é a entrada do usuário no sistema, enquanto os arquivos que vou mostrar depois são a base que mantém essa tela funcionando.

00:20 a 00:35 - Deploy do backend

Na tela: Painel do Render com o serviço backend-hub-vigilanciaepi.

Aqui aparece o serviço do backend ou hub no Render. Essa parte representa a ideia de publicar um componente do sistema fora da máquina local, para que o frontend consiga localizar e conversar com o backend disponível.

O hub ajuda a organizar os servidores de processamento, principalmente quando a aplicação precisa escolher entre mais de um backend.

00:35 a 00:50 - Transição para arquivos locais

Na tela: Você alterna de janelas até chegar ao explorador de arquivos.

Depois eu volto para os arquivos locais do projeto. Essa parte mostra onde ficam os scripts, pastas e recursos que compõem o protótipo.

A ideia não é explicar linha por linha do código, e sim mostrar que o sistema foi separado em partes: execução, interface, backend, dados e modelos.

00:50 a 01:02 - Pasta principal

Na tela: Explorador mostrando a raiz do projeto, com arquivos como iniciar, run, requirements, backend, frontend, dados e modelos.

Na pasta principal ficam os arquivos de execução e configuração. Por exemplo, existem scripts para iniciar o projeto, o arquivo de dependências e as pastas principais do sistema.

A pasta frontend guarda as telas usadas no navegador. A pasta backend concentra a API e o processamento. A pasta dados guarda bancos e arquivos gerados pelo uso do sistema. Já a pasta models guarda os pesos dos modelos de detecção.

01:02 a 01:16 - Dados do sistema

Na tela: Pasta dados e subpastas de usuários.

Aqui eu mostro a parte de dados. O protótipo cria uma estrutura separada por usuário, então cada conta pode ter seu próprio banco, seus modelos e seus metadados.

Essa organização ajuda a manter os testes separados e também prepara o projeto para um cenário em que cada empresa teria seus próprios cadastros e configurações.

01:16 a 01:28 - Backend

Na tela: Pasta backend com arquivos como app.py, processor.py, multistream.py, users.py e hub.

Na pasta backend ficam os arquivos principais da aplicação em Python. O app concentra a API, o processor fica ligado ao processamento das imagens, o multistream trata a ideia de várias câmeras, e o users organiza a parte de contas.

A pasta hub é a parte pensada para registrar backends disponíveis e permitir que a interface escolha um servidor de processamento.

01:28 a 01:36 - Modelos

Na tela: Pasta models com arquivos YOLO.

Na pasta models ficam os modelos base usados pelo protótipo, como os arquivos YOLO. Eles são o ponto de partida para detectar objetos e EPIs nas imagens.

Quando o usuário treina ou envia modelos próprios, a ideia é que o sistema também consiga trabalhar com esses modelos personalizados.

01:36 a 01:41 - Encerramento

Na tela: Volta para a gravação no OBS.

Então esse segundo vídeo complementa a demonstração: o primeiro mostra o protótipo por fora, na interface; e este mostra a estrutura por trás, com arquivos, backend, dados, modelos e deploy.

Fechamento opcional para usar no final da apresentação

Com esses dois vídeos, eu consigo mostrar tanto a experiência do usuário quanto a estrutura técnica do projeto. O Vigilância EPI ainda é um protótipo, mas já demonstra o fluxo principal: cadastrar dados, configurar câmeras, organizar modelos e preparar o processamento para apoiar a identificação de falta de EPI.

Os próximos passos seriam melhorar a precisão com datasets maiores, registrar histórico de alertas, criar permissões por perfil e reforçar as políticas de privacidade para uso em ambiente real.